

Graph Theory and Python Exercises

Algoritmos Avançados de Bioinformática

April 21, 2026

Section 1

Fundamental Graph Concepts

Basic Definitions

- **Vertex (Node):** The fundamental unit of a graph (e.g., a person in a social network).
- **Edge (Arc):** A connection between two vertices.
- **Undirected Graph:** Edges are bidirectional.
- **Directed Graph (Digraph):** Edges have a specific direction.

Neighborhoods

- **Adjacency:** Two vertices are “adjacent” if there is an edge connecting them.
- **Successor:** In a directed edge $U \rightarrow V$, V is the successor of U .
- **Predecessor:** In a directed edge $U \rightarrow V$, U is the predecessor of V .

Paths and Connectivity

- **Path:** A sequence of edges joining a sequence of distinct vertices.
- **Cycle:** A path that starts and ends at the same vertex.
- **Connectivity:** - *Undirected:* Path exists between every pair.
 - ▶ *Directed (Strong):* Path exists in both directions for all pairs.
- **Reachability:** V is reachable from U if a path exists from U to V .

Section 2

Graph Representations

Adjacency Matrix

An **Adjacency Matrix** is a 2D array of size $V \times V$ where V is the number of vertices.

- If there is an edge from vertex i to vertex j , the cell $A[i][j]$ is set to **1** (or the edge weight).
- Otherwise, the cell is set to **0**.
- **Pros:** Constant time $O(1)$ to check if an edge exists.
- **Cons:** Space complexity is $O(V^2)$, which is inefficient for “sparse” graphs (graphs with few edges).

Adjacency List

An **Adjacency List** represents a graph as a collection of linked lists or arrays.

- Each vertex has a list of all the other vertices it is connected to.
- In Python, this is usually implemented as a **Dictionary**: $G = \{0: [1, 2], 1: [2], 2: [0, 3], 3: []\}$
- **Pros:** Space efficient $O(V + E)$; faster to iterate over neighbors.
- **Cons:** Checking if a specific edge (u, v) exists takes $O(\text{degree}(u))$ time.

Comparison: Matrix vs. List

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge Lookup	$O(1)$	$O(\text{degree}(V))$
Add Vertex	$O(V^2)$	$O(1)$
Iterate Neighbors	$O(V)$	$O(\text{degree}(V))$
Best For...	Dense Graphs	Sparse Graphs

Implementation Example (Python)

Adjacency List (Common in Python)

```
adj_list = {  
    'A': ['B', 'C'],  
    'B': ['C'],  
    'C': ['A']  
}
```

Adjacency Matrix (using nested lists)

A=0, B=1, C=2

```
adj_matrix = [  
    [0, 1, 1], # A connects to B, C  
    [0, 0, 1], # B connects to C  
    [1, 0, 0]  # C connects to A  
]
```

Section 3

Structural Concepts

Degree and Neighborhoods

- **In-Degree:** The number of edges entering a vertex.
- **Out-Degree:** The number of edges leaving a vertex.
- **Isolated Vertex:** A vertex with a degree of 0.
- **Regular Graph:** A graph where every vertex has the same degree.
- **Sources and Sinks:**
 - ▶ A **Source** has an in-degree of 0.
 - ▶ A **Sink** has an out-degree of 0.

Subgraphs and Complements

- **Subgraph:** A graph formed from a subset of the vertices and edges of G .
- **Complement ($\bar{G}$):** A graph with the same vertices, but edges exist only where they *do not* exist in G .
- **Transpose Graph:** Created by reversing the direction of every edge in a directed graph.
- **Isomorphism:** Two graphs are isomorphic if they contain the same number of vertices connected in the same way, regardless of their visual “layout.”

Section 4

Connectivity and Paths

Types of Paths

- **Simple Path:** A path with no repeated vertices.
- **Shortest Path:** The path between two nodes with the minimum number of edges (or minimum total weight).
- **Eulerian Path:** A path that visits every **edge** exactly once.
- **Hamiltonian Path:** A path that visits every **vertex** exactly once.

Components and Cut-points

- **Connected Components:** Subsets of vertices where every vertex is reachable from any other in the same subset.
- **Articulation Point:** A vertex whose removal increases the number of connected components.
- **Bridge:** An edge whose removal disconnects the graph.
- **Bipartite Graph:** A graph whose vertices can be divided into two independent sets U and V such that every edge connects a vertex in U to one in V .

Section 5

Trees and Cycles

Tree Properties

A **Tree** is a special type of graph that is: 1. **Connected**: Every node can reach every other node. 2. **Acyclic**: It contains no cycles. 3. **Minimally Connected**: Removing any edge disconnects the graph.

- **Leaf Node**: A node with degree 1 (in a tree).
- **Spanning Tree**: A subgraph that is a tree and includes all vertices of G .
- **Minimum Spanning Tree (MST)**: A spanning tree where the sum of edge weights is minimized.

Directed Acyclic Graphs (DAGs)

- **DAG:** A directed graph with no cycles.
- **Topological Sort:** A linear ordering of vertices such that for every directed edge uv , vertex u comes before v .
- **Reachability:** Often calculated using the **Transitive Closure** of the graph.

Section 6

Advanced Metrics

Centrality and Clustering

- **Clustering Coefficient:** Measures how close a vertex's neighbors are to being a “clique” (fully connected).
- **Betweenness Centrality:** Measures how often a node acts as a bridge along the shortest paths between other nodes.
- **Closeness Centrality:** The reciprocal of the sum of the shortest path distances from a node to all other nodes.
- **PageRank:** An algorithm used to rank the importance of nodes based on the quantity and quality of links to them.

Network Flow

- **Capacity:** The maximum amount of “flow” an edge can handle.
- **Max Flow:** The maximum amount of data/fluid that can pass from a Source to a Sink.
- **Min-Cut:** The minimum total weight of edges that, if removed, would completely separate the Source from the Sink.
- **Max-Flow Min-Cut Theorem:** The value of the max flow is equal to the capacity of the min-cut.

Section 7

Graph Traversal Algorithms

Breadth-First Search (BFS)

BFS explores the graph layer by layer, visiting all neighbors at the current distance before moving further out.

- **Mechanism:** Uses a **Queue** (FIFO).
- **Complexity:** $O(V + E)$.
- **Key Use Cases:**
 - ▶ Finding the shortest path in **unweighted** graphs.
 - ▶ Testing if a graph is bipartite.
 - ▶ Finding all nodes within a specific distance k .

Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking.

- **Mechanism:** Uses a **Stack** (LIFO) or **Recursion**.
- **Complexity:** $O(V + E)$.
- **Key Use Cases:**
 - ▶ Detecting cycles in directed and undirected graphs.
 - ▶ Topological sorting of a DAG.
 - ▶ Finding Strongly Connected Components (SCCs).

BFS Example

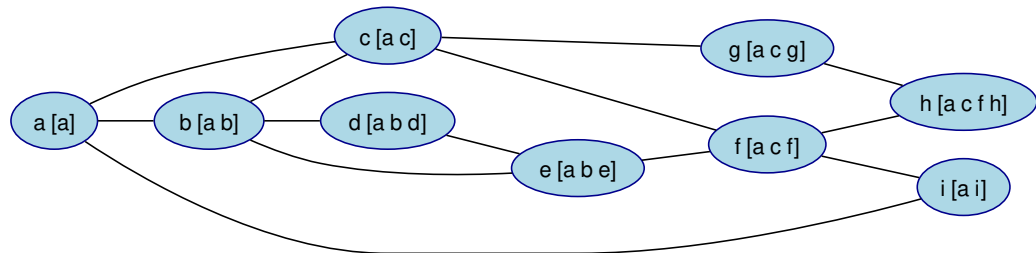


Figure 1: BFS

DFS Example

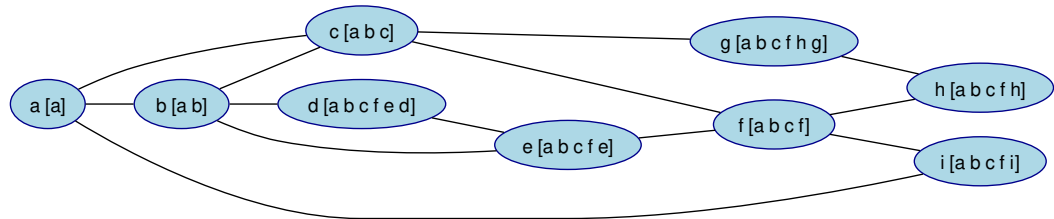


Figure 2: DFS

Section 8

Shortest Path Algorithms

Dijkstra's Algorithm

Dijkstra finds the shortest path from a single source vertex to all other vertices in a **weighted** graph.

- **Requirement:** Edge weights must be **non-negative**.
- **Mechanism:** Uses a **Priority Queue** (Min-Heap) to always expand the node with the smallest known distance.
- **Complexity:** $O((V + E) \log V)$ with a binary heap.

BFS vs. DFS vs. Dijkstra

Algorithm	Data Structure	Best For...
BFS	Queue	Shortest path (unweighted)
DFS	Stack/Recursion	Connectivity & Cycles
Dijkstra	Priority Queue	Shortest path (weighted)

Section 9

Exercises: Level 1-2

Theme 1: Basic Structure

Create the following functions where G is an adjacency list:

- ① `count_vertices(G)`: Total number of vertices.
- ② `count_edges(G)`: Total number of edges.
- ③ `get_successors(G, u)`: List of all successors of u .
- ④ `get_predecessors(G, u)`: List of all predecessors of u .
- ⑤ `is_adjacent(G, u, v)`: True if edge $u \rightarrow v$ exists.
- ⑥ `get_degree(G, u)`: Number of edges connected to u .
- ⑦ `get_in_degree(G, u)`: Incoming edges for u .
- ⑧ `get_out_degree(G, u)`: Outgoing edges for u .

Theme 1: Basic Structure

- 9 `list_isolated_vertices(G)`: Vertices with no edges.
- 10 `get_all_edges(G)`: List of tuples of all edges.
- 11 `is_regular(G)`: Check if all vertices have the same degree.
- 12 `get_sources(G)`: Vertices with in-degree 0.
- 13 `get_sinks(G)`: Vertices with out-degree 0.
- 14 `add_vertex(G, u)`: Add a new vertex to the graph.
- 15 `remove_edge(G, u, v)`: Delete the edge between u and v .

Section 10

Exercises: Level 3-4

Theme 2: Representations

- 16 `to_adjacency_matrix(G)`: Convert to 2D array.
- 17 `from_adjacency_matrix(G, matrix)`: Rebuild G from matrix.
- 18 `get_complement(G)`: Return the inverse graph.
- 19 `is_undirected(G)`: Check if every edge (u, v) has (v, u) .
- 20 `to_edge_list(G)`: Convert to list of pairs.
- 21 `is_sparse(G)`: True if $E \ll V^2$.
- 22 `transpose_graph(G)`: Reverse all edge directions.
- 23 `copy_graph(G)`: Create a deep copy.

Theme 2: Modification & Weights

- 24 `merge_graphs(G, H)`: Combine two graphs.
- 25 `subgraph(G, vertices)`: Filter G by a list of vertices.
- 26 `is_isomorphic_simple(G, H)`: Check if small graphs are identical.
- 27 `weight_edges_randomly(G)`: Assign weights to edges.
- 28 `get_heaviest_edge(G)`: Find edge with max weight.
- 29 `normalize_weights(G)`: Scale weights between 0 and 1.
- 30 `remove_self_loops(G)`: Remove edges where $u = v$.

Section 11

Exercises: Level 5-6

Theme 3: Connectivity & Paths

- 31 `has_path(G, u, v)`: Check reachability.
- 32 `find_any_path(G, u, v)`: Return any valid path.
- 33 `all_simple_paths(G, u, v)`: Find all paths (no repeats).
- 34 `find_shortest_path_unweighted(G, u, v)`: Use BFS.
- 35 `is_connected_undirected(G)`: Connectivity check.
- 36 `count_connected_components(G)`: Count islands.
- 37 `get_connected_components(G)`: Return sets of vertices.
- 38 `find_diameter(G)`: Longest shortest path.
- 39 `is_reachable_within_k(G, u, v, k)`: Check distance constraint.
- 40 `get_distance_matrix(G)`: All-pairs shortest paths.

Theme 3: Advanced Connectivity

- 41 `is_bridge(G, u, v)`: Does removal disconnect G ?
- 42 `find_articulation_points(G)`: Find critical nodes.
- 43 `is_bipartite(G)`: Can it be 2-colored?
- 44 `get_eccentricity(G, u)`: Max distance from u .
- 45 `find_radius(G)`: Minimum eccentricity.
- 46 `find_center(G)`: Vertices with min eccentricity.
- 47 `is_strongly_connected(G)`: Directed graph check.
- 48 `is_weakly_connected(G)`: Undirected version check.
- 49 `find_path_length(G, path)`: Sum of weights.
- 50 `find_all_distances_from_source(G, s)`: Dijkstra/BFS dict.

Section 12

Exercises: Level 7-8

Theme 4: Cycles & Trees

- 51 `has_cycle_undirected(G)`: Cycle detection.
- 52 `has_cycle_directed(G)`: Use DFS/Recursion stack.
- 53 `is_tree(G)`: Connected and acyclic.
- 54 `is_forest(G)`: Collection of trees.
- 55 `find_cycle_nodes(G)`: Nodes belonging to cycles.
- 56 `is_dag(G)`: Directed Acyclic Graph check.
- 57 `topological_sort(G)`: Linear ordering for DAG.
- 58 `count_leaf_nodes(G)`: Nodes with degree 1.
- 59 `find_spanning_tree(G)`: Any spanning tree.
- 60 `minimum_spanning_tree_kruskal(G)`: MST implementation.

Theme 4: Euler & Hamilton

- 01 `minimum_spanning_tree_prim(G)`: MST implementation.
- 02 `is_eulerian_path_exists(G)`: Degree parity check.
- 03 `find_eulerian_circuit(G)`: Hierholzer's or Fleury's.
- 04 `is_hamiltonian_cycle(G, path)`: Verify a cycle.
- 05 `get_tree_height(G, root)`: Max depth.
- 06 `list_ancestors(G, u, root)`: Nodes on path to root.
- 07 `find_lca(G, u, v, root)`: Lowest Common Ancestor.
- 08 `is_planar_small(G)`: Visual/Euler formula check.
- 09 `count_triangles(G)`: 3-cycles.
- 70 `find_clique_size_3(G)`: Complete subgraphs of 3.

Section 13

Exercises: Level 9-10

Theme 5: Centrality & SCCs

- 71 `betweenness centrality(G, u)`: Shortest path bridge score.
- 72 `closeness centrality(G, u)`: Distance-based score.
- 73 `k_core_decomposition(G, k)`: Iterative pruning.
- 74 `find_communities_greedy(G)`: Modularity optimization.
- 75 `is_tournament(G)`: Complete directed graph.
- 76 `transitive_closure(G)`: All reachability edges.
- 77 `get_scc_kosaraju(G)`: Two-pass SCC.
- 78 `check_2_colorability(G)`: Color assignment.
- 79 `find_maximum_clique(G)`: Largest complete subgraph.
- 80 `get_strongly_connected_components_tarjan(G)`: Single-pass SCC.

Theme 5: Algorithms & Flow

- 81 `dijkstra(G, start, end)`: Shortest path.
- 82 `bellman_ford(G, start)`: Handle negative weights.
- 83 `floyd_warshall(G)`: Matrix-based paths.
- 84 `find_negative_cycles(G)`: Detection via Bellman-Ford.
- 85 `max_flow_ford_fulkerson(G, s, t)`: Capacity networks.
- 86 `min_cut(G, s, t)`: Max-flow Min-cut theorem.
- 87 `is_bipartite_matching(G)`: Max matching.
- 88 `get_clustering_coefficient(G, u)`: Local density.
- 89 `get_average_clustering(G)`: Global density.
- 90 `page_rank_simple(G, iterations)`: Ranking nodes.

Section 14

Exercises: Level 11

Theme 6: Optimization

- 91 `traveling_salesman_brute(G)`: Optimal cycle.
- 92 `is_perfect_matching(G, matching)`: Validation.
- 93 `graph_coloring_greedy(G)`: Heuristic coloring.
- 94 `find_articulatory_bridges(G)`: Tarjan's bridge algorithm.
- 95 `get_peripheral_vertices(G)`: Nodes at diameter distance.
- 96 `is_chordal(G)`: Cycle properties.
- 97 `find_feedback_vertex_set(G)`: Minimum set to break cycles.
- 98 `contraction_hierarchy_preprocess(G)`: Shortest path prep.
- 99 `is_isomorphic_vf2(G, H)`: Advanced structure check.
- 100 `reconstruct_from_degrees(G, degree_sequence)`: Graphic sequences.